Входной файл содержит сведения о заявках на проведение мероприятий в конференц-зале. в каждой заявке указаны время начала и время окончания мероприятия (в минутах от начала суток). Если время начала одного мероприятия меньше времени окончания другого, то провести можно оба. Определите, какое максимальное количество мероприятий можно провести в конференц-зале и каков при этом максимально возможный перерыв между двумя последними мероприятиями.

Входные данные.

Задание 26

В первой строке входного файла находится число N ($N \le 1000$) — количество заявок на проведение мероприятий. Следующие N строка содержат пары чисел, обозначающих время начала и время окончания мероприятий. Каждое из чисел натуральное, не превосходящее 1440.

Запишите в ответе два числа: максимальное количество мероприятий и самый длинный перерыв между двумя последними мероприятиями (в минутах).

Пример входного файла:

```
5
10 150
100 120
131 170
150 180
120 130
```

При таких исходных данных можно провести максимум три мероприятия, например, мероприятия по заявкам 2, 3 и 5. максимальный перерыв между двумя последними мероприятиями составит 20 мин., если состоятся мероприятия по заявкам 2, 4 и 5.

Решение. Приведём решение на языке Python.

```
f = open('0.txt')
n = int(f.readline())
a=[]
for s in f:
    start,end=map(int, s.split())
    a.append([end, start])
a.sort()
count=[]
z=0
for i in range(n):
    if z==0:
        z=a[i][0]
        count.append(a[i])
        last=i
    elif z<=a[i][1]:
        z=a[i][0]
        count.append(a[i])
        last=i
del count[-1]
z=count[-1][0]
for i in range (last, n):
    if z==0:
        z=a[i][0]
        count.append(a[i])
elif z<=a[i][1]:
    count.append(a[i])
print(len(count),count[-1][1]-count[-2][0])
```

Ответ: 32 15

Во многих компьютерных системах текущее время хранится в формате «UNIX-время» — количество секунд от начала суток 1 января 1970 года.

В одной компьютерной системе проводили исследование загруженности. Для этого в течение месяца с момента UNIX-времени 1633046400 фиксировали и заносили в базу данных моменты старта и финиша всех процессов, действовавших в этой системе.

Вам необходимо определить, какое наибольшее количество процессов выполнялось в системе одновременно на неделе, начавшейся в момент UNIX-времени 1633305600, и в течение какого суммарного времени (в секундах) выполнялось такое наибольшее количество процессов.

Входные данные.

Задание 26

Первая строка входного файла содержит целое число *N* — общее количество процессов за весь период наблюдения. Каждая из следующих *N* строк содержит 2 целых числа: время старта и время завершения одного процесса в виде UNIX-времени. Все данные в строках входного файла отделены одним пробелом.

Если в качестве времени старта указан ноль, это означает, что процесс был активен в момент начала исследования. Если в качестве времени завершения указан ноль, это означает, что процесс не завершился к моменту окончания исследования.

При совпадающем времени считается, что все старты и завершения процессов происходят одновременно, в начале соответствующей секунды. В частности, если время старта одного процесса совпадает с временем завершения другого и других стартов и завершений в этот момент нет, то количество активных процессов в этот момент не изменяется.

В ответе запишите два целых числа: сначала максимальное количество процессов, которые выполнялись одновременно на неделе, начиная с момента UNIX-времени 1633305600, затем суммарное количество секунд, в течение которых на этой неделе выполнялось такое максимальное количество процессов.

Приведём другое решение на языке Python.

```
 f = open("26 (7).txt")
n = int(f.readline())
start_time = 1633305600
end_time = start_time + 604800
count = 0
time_process = [0 for i in range(0, 604801)]
for i in f:
    start_process, end_process = i.split()
    if (int(start_process) < start_time) and ((int(end_process) > start_time) or (
        count = count + 1
    if (int(start_process) >= start_time) and (int(start_process) <= end_time):
        time_process[int(start_process) - start_time] = time_process[int(start_proc
```

start_time+1

```
    if (int(end_process) >= start_time) and (int(end_process) <= end_time):
        time_process[int(end_process) - start_time] = time_process[int(end_process
sum_time = 0
max_process = 0
for i in range(1, 604801):
    count = count + time_process[i]
    if count > max_process:
        max_process = count
        sum_time = 0
    if count == max_process:
        sum_time = sum_time + 1
print(max_process, sum_time)
```

Другое решение на Python ↓

```
f = open('26.txt')
n = int(f.readline())
x0 = 1633046400
y0 = x0 + 31*24*60*60
pr = [0]*(y0-x0+1)
for i in range(n):
    a, b = map(int, f.readline().split())
    a = max(a, x0)
    if b == 0:
        pr[a-x0] += 1
    else:
        pr[a-x0] += 1
        pr[b-x0] -= 1
for i in range(1, len(pr)):
    pr[i] += pr[i-1]
x1 = 1633305600
y1 = x1 + 7*24*60*60
mx = t = 0
for i in range(x1-x0, y1-x0):
    if pr[i] > mx:
        mx = pr[i]
        t = 0
    if pr[i] == mx:
        t += 1
print(mx, t)
```

Ответ: 5000&46

3)

1)
https://disk.yandex.ru/d/ml4B4umCx1YNtw

26 На складе хранятся кубические контейнеры различного размера. Чтобы сократить занимаемое при хранении место, контейнеры вкладывают друг в друга. Один контейнер можно вложить в другой, если размер стороны внешнего контейнера превышает размер стороны внутреннего на 5 и более условных единиц. Группу вложенных друг в друга контейнеров называют блоком. Количество контейнеров в блоке может быть любым. Каждый блок, независимо от количества и размера входящих в него контейнеров, а также каждый одиночный контейнер, не входящий в блоки, занимает при хранении одну складскую ячейку.

Зная количество контейнеров и их размеры, определите минимальное количество ячеек для хранения всех контейнеров и максимально возможное количество контейнеров в одном блоке.

5)

~26

```
f = open('26.txt')
n = f.readline()
cubes = sorted([int(i) for i in f], reverse=True)
cklad=[]
while len(cubes)>0:
    block = [cubes.pop(0)]
    for i in range(len(cubes)):
        if (block[-1]-cubes[i])>=5:
            block.append(cubes[i])
            cubes[i]=''
    cubes=[x for x in cubes if x!='']
    cklad.append(block)
print(len(cklad),max(len(c) for c in cklad))
'''
```

2. https://disk.yandex.ru/d/HsSvYZZ_Z6YD-g

26 Входной файл содержит заявки на проведение мероприятий в конференц-зале в течение одного дня. В каждой заявке указано время начала и время окончания мероприятия в минутах от начала суток. Если время проведения двух или более мероприятий пересекается, то провести можно не более одного из них. Между окончанием одного мероприятия и началом следующего необходим перерыв не менее 15 минут.

Определите, какое максимальное количество мероприятий можно провести в конференц-зале в этот день и каким при этом может быть максимально возможный перерыв между двумя последними мероприятиями.

Входные данные

Первая строка входного файла содержит целое число N ($N \le 1000$) – общее количество заявок. Каждая из следующих N строк описывает одну заявку и содержит 2 целых числа, обозначающих время начала и время окончания мероприятия в минутах с начала суток. Все числа не превосходят 1440, второе число в каждой строке больше первого.

В ответе запишите два целых числа: сначала максимальное количество мероприятий, которые можно провести в этот день, затем – максимально возможный при таком количестве мероприятий перерыв между двумя последними мероприятиями.

```
f = open('26-2.txt')
n = int(f.readline())
a=[]
for s in f:
    start,end=map(int, s.split())
    a.append([end, start])
a.sort()
count=[]
z=0
for i in range(n):
    if z==0:
        z=a[i][0]
        count.append(a[i])
        last=i
    elif z<=a[i][1]:
        z=a[i][0]
        count.append(a[i])
        last=i
del count[-1]
z=count[-1][0]

for i in range (last, n):
    if z==0:
        z=a[i][0]
        count.append(a[i])
    elif z<=a[i][1]:
        count.append(a[i])
print(len(count), count[-1][1] - count[-2][0])
```

6 класс Список ре...

Дано: что-то связанное с контейнерами и хранением! Найти задание!

5

```
# 26
'''
print ('#26')
f=open('26.txt')
n=int(f.readline())
a=[]
for i in range(n):
    x=int(f.readline())
    a.append(x)
a.sort(reverse=True)
k=1
p=a[0]
for i in range(1, len(a)):
    if p-a[i]>=3:
        k=k+1
        p=a[i]
print(k, p)
```

6 - 7109 (Вопенков) Входной файл: расписан фильмов во всех кинотеатрах Москвы за весь прошедший месяц. Определить вычислив время в течение которого показывалось хотя бы 1 фильм

Решение:

```
f = open('26.txt')
n = int(f.readline())
a = []
for s in f:
    start, end = list(map(int, s.split()))
    a.append([start, end])
a.sort()
time = ['0']*44640
for i in range(n):
    for k in range(int(a[i][0]), int(a[i][1])):
        time[k]='1'
print(time.count('1'))
k=mx=0
for i in range(len(time)):
    if time[i]=='1':
        k+=1
    else:
        mx=max(mx,k)
        k=0
print(mx)
```

26

27

⑦ Демо 2024 конференция

решу ЕГЭ №9704

Дано:
5 - кол-во мероприятий
10 150
100 110 - t нач. мероприятия
t нач.

Найти: max кол-во мероприятий, которые можно провести в конференц-зале, и самое позднее время окончания последнего мероприятия.

Решение:

```
f = open('26_demo.txt')
s, n = map(int, f.readline().split())
d = (f.readlines())
a = []
for i in range(len(d)):
    a.append(int(d[i]))
a = sorted(a)[::-1]
while sum(a) >= s:
    a.pop(0)
print(len(a))
k = [int(s) for s in d]
maxk = 0
for j in range(len(k)):
    if k[j] < 53 and k[j] > 29:
        maxk = max(maxk, k[j])
print(maxk)
```

⑧ Комиков (№4933)

Автомат фиксирует пассажиров некоторого автобуса по ходу рейса. У каждого пассажира есть время входа, выхода, с момента нач. рейса.

Узнать max кол-во пассажиров одновременно находящихся в автобусе и общее время когда в автобусе был хотя бы 1 пассажир. Время входа и выхода в автобус уникально.

Решение:

7
10 10
50 130
70 130
75 90
120 170

Поиск в Libre Office Writer

```
f = open('26.txt')
n = int(f.readline())
a = []
for s in f:
    vxod, vyxod = list(map(int, s.split()))
    a.append([vxod, vyxod])
a.sort()

s = [0]*100000
for x in range(n):
    for j in range(a[x][0], a[x][1]):
        s[j] += 1
print(max(s))
k = 0
for h in range(len(s)):
    if s[h] != 0:
        k += 1
print(k)
```

(9) Задание с фасбеаки.

Решение:

```
f = open('26_demo.txt')
s, n = map(int, f.readline().split())
a = []
for i in f:
    a.append(int(i))
a = sorted(a)[::-1]
print(a)  # посмотреть промежут. рез-т
while sum(a) > s:
    a.pop(0)
print(len(a), sum(a))
```

дописано

26

27

Демо 2025 (фипи)

26

Задание выполняется с использованием прилагаемых файлов.

Во время сессии студенты сдают 4 экзамена, за каждый из которых можно получить от 2 до 5 баллов. Студенты, получившие хотя бы одну «двойку», считаются не сдавшими сессию. Результаты сессии публикуются в виде рейтингового списка, в котором сначала указаны идентификационные номера студентов (ID), сдавших сессию, в порядке убывания среднего балла за сессию, а в случае равенства средних баллов – в порядке возрастания ID. Затем располагаются ID студентов, не сдавших сессию: сначала – получивших одну «двойку», затем – две «двойки», потом ID студентов с тремя «двойками» и, наконец, ID студентов, получивших по 2 балла за каждый из экзаменов. Если студенты имеют одинаковое количество «двоек», то их ID в рейтинге располагаются в порядке возрастания.

Повышенную стипендию получают студенты, занявшие в рейтинговом списке первые 25 % мест, при условии отсутствия у них «двоек». Гарантируется, что без «двоек» сессию сдали не менее 25 % студентов. Найдите ID студента, который занимает последнее место среди студентов с повышенной стипендией, а также ID первого в рейтинговом списке студента, который имеет более двух «двоек».

В ответе запишите два целых положительных числа: сначала ID студента, который занимает последнее место среди студентов с повышенной стипендией, затем ID первого в рейтинговом списке студента, который имеет более двух «двоек».

Входные данные

В первой строке входного файла находится число N, обозначающее количество студентов (целое положительное число, не превышающее 10 000). Каждая из следующих N строк содержит 5 чисел через пробел: ID студента (целое положительное число, не превышающее 100 000) и четыре оценки, полученные им за сессию. Гарантируется, что общее число студентов N кратно 4 и хотя бы один студент имеет более двух «двоек». Во входном файле все ID различны.

Выходные данные

Два натуральных числа: искомые ID студентов в порядке, указанном в условии задачи.

Типовой пример организации данных во входном файле

```
8
4 4 4 4 4
7 5 5 5 2
10 3 4 4 5
1 4 4 4 3
6 3 5 5 3
2 2 2 2 2
13 2 2 2 3
3 3 3 3 3
```

ответ: 52326 | 635

При таких исходных данных рейтинговый список ID имеет вид: 4 6 10 1 3 7 13 2. Ответ: 6 13.

Типовой пример имеет иллюстративный характер. Для выполнения задания используйте данные из прилагаемого файла.

ДЕМОВЕРСИЯ

26

Задача 26

№26.1

При онлайн-покупке билета на концерт известно, какие места в зале уже заняты. Необходимо купить билет на такое место в ряду, чтобы перед ним как можно больше идущих подряд кресел с таким же номером было свободно. Если места, удовлетворяющие этому условию, есть в нескольких рядах, то нужно выбрать ряд, расположенный как можно ближе к сцене. В ответе запишите два целых числа: искомый номер ряда и количество свободных кресел перед выбранным местом. Нумерация рядов и мест ведётся с 1. Гарантируется, что хотя бы одно такое место в зале есть.

Входные данные

В первой строке входного файла находятся три числа: N - количество занятых мест в зале (целое положительное число, не превышающее 10 000), M - количество рядов (целое положительное число, не превышающее 100 000) и K - количество мест в каждом ряду (целое положительное число, не превышающее 100 000). В следующих N строках находятся пары натуральных чисел – номер ряда и номер места занятого кресла соответственно (первое число в паре не может принимает значения M, а второе - K).

Выходные данные

Два целых положительных числа: искомый номер ряда и количество свободных кресел перед выбранным местом.

Решение

```
File = open("26.txt")
N, M, K = map(int, File.readline().split())
places = []  # Список для считывания занятых мест
for i in range(N):
        row, place = map(int, File.readline().split())
        places.append([place, row])  # На первое место ставим место, потом ряд
        # Такое расположение обеспечивает сортировку именно по номеру места
places.sort()
# Максимальное количество свободных мест перед искомым свободным местом
```


Сайт 100ballnik.com - варианты формата ОГЭ и ЕГЭ

```
maxCount = 0
# Минимальный номер ряда такого места
minRow = 0
for p in range(1, N):
        previousPlace = places[p - 1][0]  # Номер прошлого места
        previousRow = places[p - 1][1]  # Номер ряда прошлого места
        currentPlace = places[p][0]  # Номер текущего места
        currentRow = places[p][1]  # Номер ряда текущего места
        # Если номера совпадают, то текущее место стоит после прошлого
        if currentPlace == previousPlace:
            # Считаем кол-во мест между местом перед предыдущим местом
            count = (currentRow - 1) - (previousRow + 1)
            # Если максимум
            if maxCount <
                maxCount =
                minRow = currentRow - 1
            # Если максимум остался тот же, можно ли уменьшить номер ряда
            elif maxCount == count:
                minRow = min(minRow, currentRow - 1)

        elif currentPlace - previousPlace == 1:
            # Количество мест от конца зала до текущего места
            count = M - (previousRow + 1)
            if maxCount < count:
                maxCount = count
                minRow = M
            elif maxCount == count:
                minRow = min(minRow, M)

            # Количество мест от предыдущего места до стены
            count = currentRow - 2
            if maxCount < count:
                maxCount = count
```

```
            minRow = currentRow - 1
        elif maxCount == count:
            minRow = min(minRow, currentRow - 1)

    else:  # Есть полностью пустой ряд
        maxCount = K - 1  # Кол-во свободных мест это K-1
        minRow = M  # Минимальный ряд в таком случае - последний
        # Дальше не нужно перебирать, максимальное кол-во не поменяется
        break

print(minRow, maxCount)
```

Решение экселем:

Если мы знаем, что место 1 занято в ряду 1 и 6, значит между ними есть ряды 2, 3, 4, 5, которые свободны. А если они свободны, значит перед местом в 5-м ряду есть три свободных места. Найти можем как $(6 - 1) - (1 + 1) = 3$.

Если же это место первое занято, например в ряду 4, то перед ним всего $4 - 1 = 3$ свободных мест, значит перед местом в 3-м ряду мест $4 - 2 = 2$.

Также, если это первое место, то нужно от предыдущего рассматриваемого также посчитать количество мест до конца зала, так как последнее место в «столбце» мест тоже может считаться. Тогда, если предыдущее последнее место занято в ряду 4, количество мест это $6 - (4 + 1)$.

Чтобы это посчитать в экселе, отсортируем по возрастанию сначала столбец B, потом столбец A.

Сайт 100ballnik.com - варианты формата ОГЭ и ЕГЭ

	A	B
1	1	1
2	6	1
3	5	2
4	4	3
5	2	4
6	5	5
7	3	6
8		6
9		7

В Ячейку C2 запишем 0.

В ячейку C3 вставим формулу и растянем до конца:

$$= \text{ЕСЛИ}(B2 = B3; \text{МАКС}((A3 - 1) - (A2 + 1); C2); A3 - 2)$$

Чтобы определить, можно ли выбрать место в конце зала, в ячейку $D2$ запишем формулу и растянем до конца:

$$= \text{ЕСЛИ}(B3 <> B2; 6 - (A2 + 1); 0)$$

Чтобы определить максимальное кол-во мест в этом «столбце» мест с одинаковым номером, запишем в ячейку $E2$ формулу и растянем до конца:

$$= \text{МАКС}(C2 : D2)$$

Чтобы определить номер ряда, в ячейку $F2$ запишем формулу и растянем до конца:

$$= \text{ЕСЛИ}(E2 = \text{МАКС}(E : E); A2 - 1; 9999999)$$

Если текущее кол-во свободных мест — это максимум столбца - выводим номер ряда -1, иначе что-то большое.

Сайт 100ballnik.com - варианты формата ОГЭ и ЕГЭ

Окончательный номер ряда узнаем, если сравним для каждого места максимальное перед ним и после него, если после – больше, тогда минимальный номер ряда это самый последний ряд. В ячейку $G2$ запишем и растянем формулу:

$$= \text{ЕСЛИ}(C2 >= D2; F2; 6)$$

Таким образом нужно выбрать строчку с минимальным числом из столбца G, в ней в столбце G - первый ответ, в столбце F - второй.

	A	B	C	D	E	F	G
1	Ряды	Места	Места перед	Места после	Максимальное кол-во мест	Минимальный Номер ряда	
2	1	1	0	4	0	9999999	9999999
3	6	1	3	-1	3	5	5
4	5	2	3	0	3	4	4
5	4	3	2	1	2	9999999	9999999
6	2	4	0	3	3	1	6
7	5	5	3	0	3	4	4
8	3	6	1	0	1	9999999	9999999
9	6	6	1	-1	1	9999999	9999999
10	4	7	2	1	2	9999999	9999999

№26.2 (от 8.06)

Задание №26 При онлайн-покупке билета на концерт известно, какие места в зале уже заняты. Необходимо купить два билета на такие соседние места в одном ряду, чтобы перед ними все кресла с такими же номерами были свободны, а ряд находился как можно дальше от сцены. Если в этом ряду таких пар мест несколько, найдите пару с наибольшими номерами. В ответе запишите два целых числа: искомый номер ряда и наибольший номер места в найденной паре. Нумерация рядов и мест ведётся с 1. Гарантируется, что хотя бы одна такая пара в зале есть.

Входные данные:

В первой строке входного файла находятся три числа: N - количество занятых мест в зале (целое положительное число, не превышающее 10 000), M - количество рядов (целое положительное число, не превышающее 100 000) и K - количество мест в каждом ряду (целое положительное число, не превышающее 100 000). В следующих N строках находятся пары натуральных чисел: номер ряда и номер места занятого

Сайт 100ballnik.com - варианты формата ОГЭ и ЕГЭ

кресла соответственно (первое число не превышает значения M, а второе - K).

Выходные данные:

Два целых положительных числа: наибольший номер ряда и наибольший номер места в найденной паре кресел. Типовой пример организации данных во входном файле:

7 7 8
1 1
6 6
5 5
6 7
4 4
2 2
3 3

При таких исходных данных ответом является пара чисел 5 и 8. Условию задачи удовлетворяют места 7 и 8 в ряду 5, перед креслами 7 и 8 нет занятых мест и это последняя из двух возможных пар в этом ряду. В рядах 6 и 7 искомую пару найти нельзя.

Идея решения

Так как нам нужны ВСЕ свободные места ПЕРЕД креслом, нужно "отфильтровать оставив только данные по местам, которые первые в своих столбцах То есть условно есть место 2 5 и 4 5 Перед местом 4 5 уже есть 2 5, поэтому его не берём Потом пройтись по всем таким местам и выбрать те, которые в одном ряду и рядом

№26.3 (от 8.06)

Отбор абитуриентов в вуз происходит по сумме баллов трех экзаменов: по русскому языку, математике и информатике. На заранее известное количество мест зачисляются абитуриенты, набравшие большую сумму баллов по результатам трех экзаменов. Все абитуриенты, набравшие определенную сумму баллов или больше, зачисляются на имеющиеся места. Такой балл называется проходным. Если после заполнения имеющихся мест абитуриентами с проходным баллом остаются незаполненные места, но абитуриентов, набравших следующую сумму баллов, больше, чем вакантных мест, набранная этими абитуриентами сумма баллов называется полупро-

Сайт 100ballnik.com - варианты формата ОГЭ и ЕГЭ